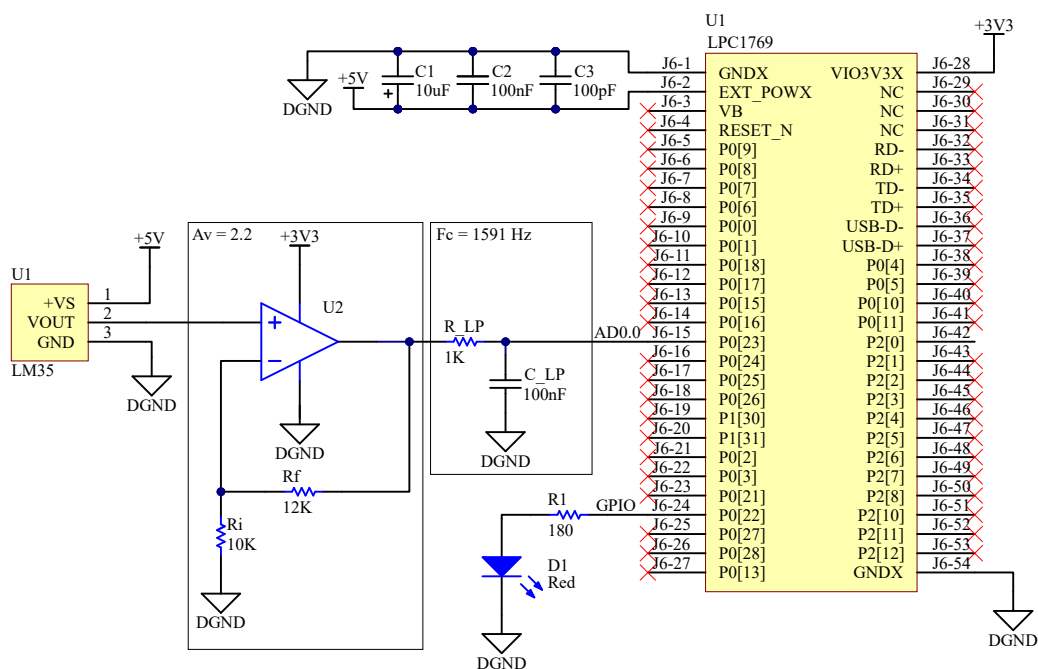


```

1  /*=====
2  * @file     Ejercicio_6_1_1.c
3  *
4  * @brief     Se requiere monitorizar la temperatura ambiente mediante un sensor
5  *            analógico LM35 conectado al canal AD0.0 del ADC..
6  *            Realizar el muestreo periódico de la señal y la detección de
7  *            sobrepaso de un umbral crítico de temperatura de 60 °C mediante una
8  *            alarma técnica visual de prioridad alta, conformada por un LED rojo
9  *            en el pin P0.22.
10 *
11 * @details   - Las señales de los sensores LM35 se acondicionan mediante
12 *             amplificadores operacionales configurados para maximizar el
13 *             aprovechamiento del rango dinámico del ADC.
14 *             - El acondicionamiento adapta la tensión generada por los
15 *             sensores al rango de entrada del LPC1769 (0 V a 3.3 V),
16 *             incrementando la resolución efectiva de la medición y
17 *             mejorando la inmunidad al ruido durante la adquisición.
18 *             - Base de tiempo de adquisición: 1 muestra por segundo (1 Hz)
19 *             mediante el disparo por hardware de MAT0.1 del Timer 0.
20 *             - Implementación en bajo nivel mediante estructuras CMSIS (NXP LPC17xx).
21 *
22 * @author    Ing. E. Migliore
23 *
24 * @version   3.0
25  */=====

```



$$A_V = \frac{V_{out(max)}}{V_{in(max)}} = \frac{V_{out(max)}}{T_{LM35(max)} S_{LM35}} = \frac{3.3 \text{ V}}{150 \text{ }^{\circ}\text{C} * 10 \frac{\text{mV}}{^{\circ}\text{C}}} = 2.2$$

$$A_V = 1 + \frac{R_f}{R_i} \Rightarrow R_f = (A_V - 1)R_i = (2.2 - 1) * 10 \text{ k}\Omega = 12 \text{ k}\Omega$$

```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5  #include <stdbool.h>
6
7  /*=====
8  * DEFINICIÓN DE CONSTANTES
9  *=====*/
10 /* Parámetros del ADC */
11 #define ADC_VREF_MV          3300U      // Tensión de Referencia ADC: 3.3 V
12 #define ADC_MAX_COUNTS      4095U      // Resolución ADC: 12 bits (2^12 - 1)
13 #define PCLK_ADC_FREQ_HZ    12500000U  // PCLK_ADC = CCLK/8 = 12.5 MHz @ CCLK = 100 MHz
14 #define ADC_SAMPLE_RATE_HZ  200000U    // ADC Conversion Rate: 200 kS/s
15 /* Implementación:
16  * CLKDIV = (PCLK_ADC / (65 * ADC_SAMPLE_RATE_HZ)) - 1
17  */
18 #define ADC_CLKDIV_VALUE \
19     ((PCLK_ADC_FREQ_HZ / (65U * ADC_SAMPLE_RATE_HZ)) - 1U)
20
21 /* Parámetros de Adquisición */
22 #define PCLK_FREQ_HZ        25000000U  // PCLK = 25 MHz (Default @ CCLK = 100 MHz)
23 #define ADC_TRIGGER_RATE_HZ  1U         // FTRIGGER = 1 Hz (1 muestra por segundo)
24 #define TMR0_PR_VALUE       0U         // PR = 0 -> 1 tick = 40 ns @ PCLK = 25 MHz
25 // Implementación de MR = ((1 / ADC_TRIGGER_RATE_HZ) / 2) / ((PR + 1) / PCLK_FREQ_HZ) - 1
26 #define TMR0_MR_VALUE       ((PCLK_FREQ_HZ / (2U * ADC_TRIGGER_RATE_HZ * (TMR0_PR_VALUE
+ 1U))) - 1U)
27
28 /* Parámetros del Sensor LM35 */
29 #define LM35_SENSITIVITY_MV  10U        // Sensibilidad: 10 mV/°C
30 #
31 /* Modalidades de Salida External Match Control - Timer */
32 #define EMC_DO_NOTHING       0x00U      // Do Nothing
33 #define EMC_CLEAR            0x01U      // Clear MATn.m output to LOW
34 #define EMC_SET               0x02U      // Set MATn.m output to HIGH
35 #define EMC_TOGGLE           0x03U      // Toggle MATn.m output
36
37 /* Umbrales de supervisión */
38 #define ALARM_TEMP_C         60U        // Umbral de alarma por sobretemperatura: 60 °C
39
40 /*=====
41 * DEFINICIÓN DE VARIABLES GLOBALES
42 *=====*/
43 volatile uint16_t adc0_raw    = 0;
44 volatile uint32_t adc0_mv     = 0;
45 volatile uint32_t adc0_temp_c = 0;
46
47
48 /*=====
49 * PROTOTIPOS DE FUNCIONES
50 *=====*/
51 void cfgGPIO(void);
52 void cfgTMR0(uint32_t period_ticks, uint32_t match_value, uint8_t mode);
53 void cfgADC(uint32_t);
54
55 /*=====
56 * FUNCIÓN PRINCIPAL
57 *=====*/
58 int main(void) {
59
60     /*-----
61     * INICIALIZACIÓN DEL MCU
62     *-----*/
63     cfgGPIO();
64     cfgTMR0(TMR0_PR_VALUE, TMR0_MR_VALUE, EMC_TOGGLE);
65     cfgADC(ADC_SAMPLE_RATE_HZ);
66
67     /*-----
68     * INICIO PROGRAMA PRINCIPAL

```

```

69     *-----*/
70     while(1){}
71
72     return 0;
73 }
74
75 /*=====
76  * DEFINICIONES DE FUNCIONES
77  *=====*/
78 /*****
79  * @brief    Configura las direcciones GPIO, modos de función y resistencias.
80  *
81  * @details  - P0.22: Función GPIO como Salida Digital (Alarma de prioridad alta)
82  *            - P0.23: Función AD0.0 como Entrada Analógica (Sensor LM35)
83  *            - P1.29: Función MAT0.1 como Salida (Match Timer 0).
84  *
85  * @param    None
86  * @return   None
87  *****/
88 void cfgGPIO(void)
89 {
90     LPC_PINCON->PINSEL1  &= ~(0b11 << 12);    // P0.22 con Funcionalidad GPIO
91     LPC_GPIO0->FIODIR    |= (1<<22);           // P0.22 como Salida Digital
92
93     LPC_PINCON->PINSEL1  &= ~(0b11 << 14);
94     LPC_PINCON->PINSEL1  |= (0b01 << 14);    // P0.23 con Funcionalidad AD0.0 (ADC)
95     LPC_PINCON->PINMODE1 &= ~(0b11 << 14);
96     LPC_PINCON->PINMODE1 |= (0b10 << 14);    // P0.23 con Modalidad Tristate (Flotante)
97
98     LPC_PINCON->PINSEL3  &= ~(0b11 << 26);
99     LPC_PINCON->PINSEL3  |= (0b11 << 26);    // P1.29 con Funcionalidad MAT0.1 (TMR0)
100
101     return;
102 }
103
104 /*****
105  * @brief    Configura e inicializa el periférico TIMER0 en modo Timer.
106  *
107  * @details  Genera una señal periódica en MAT0.1 destinada al disparo
108  *            por hardware del ADC.
109  *
110  *            La frecuencia de la señal generada depende de la base de tiempo
111  *            configurada mediante el registro Prescale Register (PR), del
112  *            valor de coincidencia MR1 y de la modalidad de salida
113  *            seleccionada para MAT0.1.
114  *
115  * @param    <period_ticks> Valor de recarga del Prescale Register (PR),
116  *            expresado en ciclos de PCLK.
117  * @param    <match_value> Valor límite del registro de coincidencia MR1.
118  * @param    <mode>        Modalidad de salida External Match Control (EMC1)
119  *            aplicada sobre MAT0.1.
120  *
121  * @return   None.
122  *****/
123 void cfgTMR0(uint32_t period_ticks, uint32_t match_value, uint8_t mode)
124 {
125     // Inicialización del TMR0
126     LPC_SC->PCONP |= (1 << 1);
127     LPC_SC->PCLKSEL0 &= ~(3 << 2);
128     LPC_TIM0->CTCR &= ~(3 << 0);
129     LPC_TIM0->PR = period_ticks;
130     LPC_TIM0->TCR |= (1 << 1);
131     LPC_TIM0->TCR &= ~(1 << 1);
132     LPC_TIM0->IR |= (1 << 0);
133
134     // Configuración del TMR0
135     LPC_TIM0->MR1 = match_value;
136     LPC_TIM0->MCR &= ~(1 << 3);
137     LPC_TIM0->MCR |= (1 << 4);

```

```

138     LPC_TIM0->MCR &= ~(1 << 5);
139
140     LPC_TIM0->EMR &= ~(0x03 << 6);
141     LPC_TIM0->EMR |= ((mode & 0x03) << 6);
142
143     NVIC_DisableIRQ(TIMER0_IRQn);
144
145     LPC_TIM0->TCR |= (1 << 0);
146
147     return;
148 }
149
150 /*****
151  * @brief   Configura e inicializa el periférico ADC.
152  *
153  * @details Configura la conversión del canal AD0.0 disparada por flanco
154  *           descendente en el hardware MAT0.1 del Timer 0.
155  *
156  * @param   <rate> Tasa de conversión deseada del ADC [S/s].
157  *
158  * @return  None
159  *****/
160 void cfgADC(uint32_t rate){
161     uint32_t clkdiv;
162     // Cálculo del divisor del reloj ADC:
163     clkdiv = (PCLK_ADC_FREQ_HZ / (65U * rate)) - 1U;
164
165     // Encendido del Periférico (AD0CR ON) - [Default: OFF]
166     LPC_SC->PCONP |= (1 << 12);
167     // Habilitación del Periférico (AD0CR ON): (PDN=1)
168     LPC_ADC->ADCR |= (1 << 21);
169     // Selección de PCLK_ADC = CCLK / 8 @ CCLK = 100 MHz → PCLK_ADC = 12.5 MHz
170     LPC_SC->PCLKSEL0 |= (3 << 24);
171     // Configuración del divisor del reloj ADC
172     LPC_ADC->ADCR &= ~(0xFF << 8);
173     LPC_ADC->ADCR |= ((clkdiv & 0xFF) << 8);
174     // Deshabilitación del Modo Burst para ADC: (BURST=0)
175     LPC_ADC->ADCR &= ~(1 << 16);
176     // Inicialización de Conversión debido a MAT0.1: (START=100)
177     LPC_ADC->ADCR &= ~(7 << 24);
178     LPC_ADC->ADCR |= (4 << 24);
179     // Inicialización de Conversión ante Flanco Descendente de MAT0.1: (EDGE=1)
180     LPC_ADC->ADCR |= (1 << 27);
181     // Funcionalidad de P0.23 como AD0.0 (Canal 0)
182     LPC_ADC->ADCR |= (1 << 0);
183
184     // Habilitación de Interrupción ante Finalización de Conversión por AD0.0
185     LPC_ADC->ADINTEN |= (1 << 0);
186     // Habilitación de Interrupción por ADC0
187     NVIC_EnableIRQ(ADC_IRQn);
188     return;
189 }
190
191 /*****
192  * @brief   Rutina de Servicio de Interrupción (ISR) para el vector ADC.
193  *
194  * @details Implementa una secuencia de adquisición de una etapa:
195  *           1. Lee el canal ADC asociado al sensor.
196  *           2. Convierte el resultado digital a unidades eléctricas.
197  *           3. Convierte la magnitud eléctrica a unidades físicas.
198  *           4. Procesa la variable adquirida y actualiza la alarma.
199  *
200  * @param   None.
201  * @return  None.
202  *****/
203 void ADC_IRQHandler(void)
204 {
205     // Espera Fin de Conversión de AD0.0: DONE_CH0 = 1?
206     if (LPC_ADC->ADDR0 & (1 << 31)){

```

```
207 // Guarda Dato Digitalizado de AD0.0 (12 bits) en Cuentas de AD
208 adc0_raw = (LPC_ADC->ADDR0 >> 4) & 0xFFF;
209
210 // Conversión de Cuentas AD a Tensión de Entrada [mV]
211 adc0_mv = (uint32_t)(adc0_raw * ADC_VREF_MV) / ADC_MAX_COUNTS;
212
213 // Conversión de Tensión de Entrada a Temperatura [°C]
214 adc0_temp_c = adc0_mv / LM35_SENSITIVITY_MV;
215
216 // Procesamiento del Dato Acondicionado en Unidad Física [°C]
217 if (adc0_temp_c > ALARM_TEMP_C) {
218     LPC_GPIO0 -> FIOSET = (1 << 22);
219 } else {
220     LPC_GPIO0 -> FIOCLR = (1 << 22);
221 }
222 }
223
224 return;
225 }
```